INTELLIGENT SYSTEMS

ESCOLA TÈCNICA SUPERIOR D'ENGINYERIA INFORMÀTICA

Lab assignment 1:

"Problem solving based on state-space search"

DEPARTAMENT DE SISTEMES INFORMÀTICS I COMPUTACIÓ
UNIVERSITAT POLITÈCNICA DE VALÈNCIA

CONTENTS

1. Introduction……*page 3*

2. General aspects of the puzzle game.……*page 3*
2.1 Representation
2.2 Operators

3. Search strategies……*page 4*

4. Description of the code provided..……*page 5*
4.1 Detailed description of "search.py"
Provided heuristic functions
Functions for interpretation and visualization
Generating puzzle states
Auxiliary methods for the execution of algorithms
Representation of the states and execution of the algorithms

5. How to add a new heuristic……*page 13*

6. Work to be done by the student……*page 15*

Appendix. How to use Jupyter Notebook or Google Colab……*page 17*

1. Introduction

The aim of this lab assignment is to evaluate the efficiency, the time cost and the space cost of several search strategies when applied to the n-puzzle problem. To this purpose, a library "search.py" is provided implementing several algorithms that we have studied in our theory lessons, as well as a number of support methods for the execution and evaluation of the proposed solutions. Several Jupyter notebooks are also provided, in which a set of exercises is proposed.

2. General aspects of the puzzle game

This section briefly summarizes the main features of the puzzle game.

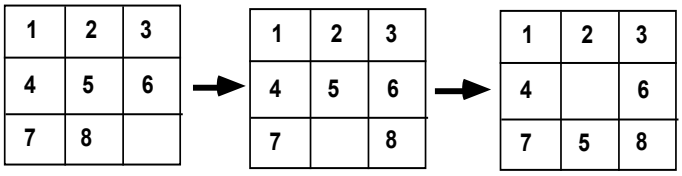
2.1. Representation

The puzzle game is represented on a board made up of n*n squares.

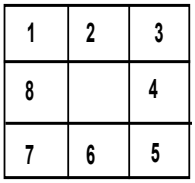
Focusing on a specific example, let us imagine an instance of the game where n=3, and so, there are 9 squares. 8 of the squares contain a tile which can be slid horizontally and vertically. Each tile is numbered from 1 to 8. There is an empty square in the board which allows the tiles to be moved.

The total number of states or configurations of the problem that can be generated in this puzzle game is 9! = 362 880 states.

Here is an example of a sequence of movements:



The **goal** of the problem is to obtain, as a solution, the sequence of "movements of the empty square" that brings us from the initial state to the goal state:



2.2. Operators

The operators in the puzzle game represent the "movements" of the empty square. These movements are: up, down, right, left. Note that any 4 of the movements is possible when the empty square is in the centre of the board; when the empty square is in a corner, then 2 of the movements are possible; in any other case, 3 of the movements are possible.

3. Search strategies

The search strategies implemented in the *puzzle* program are:

BFS. The Breadth-First Search strategy uses $f(n)=g(n)$ for node expansion, where $g(n)$ is the cost factor linked to the depth of node n in the search tree.

DFS (Backtracking). This strategy implements Depth-First Search with backtracking. To this end, it uses the function $f(n)=-g(n)$, where $g(n)$ is the cost factor linked to the depth of node n in the search tree. Unlike DFS (Graph Search), this strategy only keeps in memory the explored nodes that belong to the current path (PATH list); the rest of the explored nodes are not kept in the CLOSED list, but are deleted instead.

Greedy (Manhattan). This is a strategy that implements a *greedy algorithm* following the function $f(n)=D(n)$, where $D(n)$ is the Manhattan distance.

ID. This strategy implements an Iterative Deepening search. It involves performing successive depth-first searches with backtracking until the solution is reached. Each new search increases the depth of the exploration by 1.

A* Manhattan. This is an A-type search that uses the Manhattan distance as a heuristic; node expansion is performed according to the function $f(n)=g(n)+h(n)$, where $g(n)$ is the cost factor corresponding to the depth of node n in the search tree and $h(n)=D(n)$ is the Manhattan distance, i.e., the sum of the horizontal and vertical distances of each puzzle tile from its current position to the position it will take in the goal.

A* (MD + LC). This strategy implements an A-type search that uses an informed heuristic based on the combination of the Manhattan distance and linear conflicts (LC). Node expansion is performed using the function $f(n)=g(n)+h(n)$, where $g(n)$ is the cost associated with the depth of node n and $h(n)=D(n)+LC(n)$, where $D(n)$ is the Manhattan distance and $LC(n)$ is the number of conflicts between misplaced tiles in the same row or column that must reverse their position with each other. This heuristic improves accuracy over A* Manhattan, as it adds penalties for conflicts that involve additional necessary movements.

IDA* Manhattan. This search strategy implements an Iterative Deepening A* search where the search is bounded by the f-value, i.e., by the function $f(n)=g(n)+h(n)$, where $g(n)$ is the cost factor and $h(n)=D(n)$ is the Manhattan distance. The bound of an iteration i of the IDA* algorithm is the smallest f-value of any node that exceeds the bound in the previous iteration $i-1$.

IMPORTANT NOTE:

The DFS (Backtracking) strategy works with a maximum depth. If the current maximum depth is less than the depth of the optimal solution, then the strategy will not find a solution.

4. Description of the code provided

4 files are provided that make up the setup necessary to carry out the assignment:

1. **`search.py`**
   This Python source file contains the implementation of several classic search algorithms (such as BFS, DFS, A*, IDA*) and heuristics (such as Manhattan). It also includes auxiliary functions to visualize states, display solution paths, and compare results between algorithms.
2. **`puzzle-p1.ipynb`**
   Jupyter Notebook introducing the search algorithms applied to the puzzle problem. Its goal is to familiarize users with the use of the file search.py, the execution of search strategies, and the visualization of solutions.
3. **`puzzle-p2.ipynb`**
   Jupyter Notebook focused on how to implement and test additional heuristics, as well as on comparing heuristics. It is the main notebook for developing and validating the additional heuristic functions expected as student work in this lab assignment.
4. **`alg-compare-p3.ipynb`**
   More advanced Jupyter Notebook aimed at making detailed comparisons between heuristics. It includes graphical visualization and quantitative analysis to support the justification of results.

4.1 Detailed description of "search.py"

The file search.py contains the implementation of several classic search algorithms along with some auxiliary methods. **This file is not to be modified**.

The methods that can be used during this lab assignment are described below:

Provided heuristic functions

`getManhattanDistance(state, end_state)`

Computes the **total Manhattan distance** between the current state and the goal state. It is an admissible heuristic for n-puzzle-type problems.

- **Input parameters**:
  - `state`: list representing the current state of the board (e.g., `[1, 2, 3, 4, 5, 6, 7, 8, 0]`).
  - `end_state`: list with the goal state.

- **Returns**:
  An integer representing the sum of the Manhattan distances of all tiles (except 0/empty) between their current position and the goal position.

`getLinear_conflict(state, end_state)`

Computes the **number of linear conflicts** in rows and columns between state and end_state. A linear conflict occurs when two tiles are in the same row or column as their destination, but in the wrong order. Each selected conflict adds 2, but a tile cannot participate in more than one conflict within the same row or column.

- **Input parameters**:
  - `state`: list representing the current state of the board.
  - `end_state`: list representing the goal state.
- **Returns**:
  An integer representing the total number of linear conflicts detected, multiplied by 2.

`get_md_plus_linear_conflict(state, end_state)`

Combines two heuristics: **Manhattan distance** and **linear conflicts**. It is a more informed heuristic than Manhattan alone, and it remains admissible.

- **Input parameters**:
  - `state`: list with the current state,
  - `end_state`: list with the goal state.
- **Returns**:
  An integer: `getManhattanDistance + 2 * getLinear_conflict`.

Functions for interpretation and visualization

`check_consistency(heuristic, initial_state, end_state, size, max_states=10000)`

Performs an empirical consistency check for a heuristic function. Starting from an initial state, it traverses reachable states and verifies, for each transition between a state n and one of its successors n', that the following condition holds:

$$h(n) \leq 1 + h(n')$$

The value 1 corresponds to the cost of performing a move in the n-puzzle.

(function `check_consistency`; continuation)

- **Input parameters:**
  - `heuristic`: heuristic function to be checked. It must accept the arguments (state, end_state).
  - `initial_state`: list representing the state from which exploration begins.
  - `end_state`: list representing the goal state.
  - `size`: board dimension, for example, 3 for a 3×3 puzzle.
  - `max_states`: maximum number of states to check. Default value is 10000. None can be used to explore all reachable states.

- **Returns:**
  A dictionary with the following keys:
  - **consistent**: indicates the result of the check:
    - False if a counterexample was found.
    - None if no counterexample was found, but the max_states limit was reached.
    - True if all reachable states were explored without finding counterexamples.
  - **complete**: indicates whether the exploration of all reachable states was completed.
  - **states_checked**: number of states examined.
  - **counterexample**: contains the transition that breaks the consistency condition, or None if no trangression was found.

  When a counterexample is found, this field includes:
  - **state**: state n.
  - **child**: successor state n'.
  - **h_state**: value h(n).
  - **h_child**: value h(n').
  - **step_cost**: transition cost, which is 1 in this problem.

Usage example:

```
result = check_consistency(
    new_heuristic,
    initial_state,
    end_state,
    size,
    max_states=10000
)
print(result)
```

If a transgression is found, the result will have a structure similar to:

```
{
    'consistent': False,
    'complete': False,
    'states_checked': 25,
    'counterexample': {
```

```
            'state': [1, 2, 3, 4, 5, 6, 0, 7, 8],
            'child': [1, 2, 3, 4, 5, 6, 7, 0, 8],
            'h_state': 4,
            'h_child': 2,
            'step_cost': 1
        }
    }
```

In this case, the condition **4 ≤ 1 + 2** is broken, so the heuristic is not consistent.

`visualize_state(state)`

Displays on the command line the board state in matrix format for easy human reading.

- **Input parameters**:
  - `state`: list of integers representing the state of the puzzle.
- **Output**: Prints the board on the command line.

`show_path_for_algorithm(results, algorithm_name, size)`

Searches a dictionary of results for the information of a specific algorithm and displays its solution path.

- **Input parameters**:
  - `results`: a dictionary associating the name of each algorithm with its results.
  - `algorithm_name`: name of the algorithm the result of which we want to display.
  - `size`: board dimension.
- **Output**: Prints on the command line the steps of the corresponding solution.

`plot_algorithm_comparison(results)`

Plots a comparison of algorithms based on execution time, nodes generated, cost, etc.

- **Input parameters**:
  - `results`: dictionary with results for several algorithms (see the standard structure of the dictionary `results` in `run_algorithm`).
- **Output**:
  Plot generated using `matplotlib`.

`show_results(results)`

Prints on the command line a summary of the results obtained by each algorithm (time, depth, nodes, etc.).

- **Input parameters**:
  - `results`: dictionary with the execution results for several algorithms (see the standard structure of the dictionary `results` in `run_algorithm`).

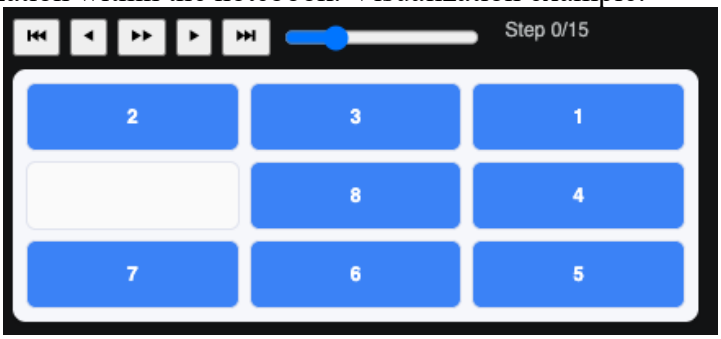
- **Output**:
  Text table with the comparative data.

`show_path_puzzle(results, algorithm_name, size, width=420)`

Interactively displays the solution path of an algorithm within a notebook using a board rendered with HTML, CSS, and JavaScript. It allows stepping forward and backward, jumping to the first or last state, playing or pausing the sequence, and adjusting the playback speed. At each step, the moved tile is highlighted, and the solution progress is indicated.

- **Input parameters:**
  - `results`: dictionary containing execution results; it must include the solution path under `results[algorithm_name]['path']`.
  - `algorithm_name`: string specifying the name of the algorithm whose path is to be visualized; it must match a key in `results`.
  - `size`: board dimension (e.g., 3 for a 3x3 puzzle).
  - `width`: maximum width of the visualization in pixels (optional; defaults to 420).

- **Output**:
  Interactive visualization within the notebook. Visualization example:



Generating puzzle states

`generate_random_state(size, end_state)`

Generates a **valid (solvable) random state** of the puzzle, based on the goal end state. This function applies random moves on the goal state without repeating states, ensuring that the result is achievable.

- **Input parameters**:
  - `size`: board dimension (e.g., `3` for an 8-tile puzzle).
  - `end_state`: list representing the goal state (e.g., sorted from 1 to n^2-1 and 0 at the end).
- **Returns**:
  List of integers representing a new state of the puzzle, random but solvable.
- **Example of use**:

```
goal = generate_ordered_state(3)
```

```
initial = generate_random_state(3, goal)
```

`generate_ordered_state(n)`

Creates the **standard ordered state** of the puzzle, where the tiles are arranged from smallest to largest, with the empty square (`0`) in the last position.

- **Input parameters**:
  - `n`: board dimension (e.g., `3` for a 3x3 board).
- **Returns**:
  List of integers of size `n * n`, ordered (e.g., `[1, 2, 3, 4, 5, 6, 7, 8, 0]` for `n=3`).

`generate_spiral_state(n)`

Generates a goal state **with spiral ordering (snail mode)**, useful as an alternative to the classic order. The tiles are arranged in a spiral from the top left corner to the centre.

- **Input parameters**:
  - `n`: board dimension (`n x n`).
- **Returns**:
  List of integers of size `n * n`, arranged in spiral order, ending with `0` at the centre (if `n` is odd).
- **Return example for `n=3`**: (actually returns a list)

```
[1, 2, 3,
 8, 0, 4,
 7, 6, 5]
```

Additionally, `search.py` includes a common registry of algorithms and heuristics.

Auxiliary methods for the execution of algorithms

`run_algorithm(algorithm_name, initial_state, end_state, size, depth=50, heuristic_func=None)`

This method executes a single search algorithm specified by its name, on an initial state and a goal state of the puzzle. It returns a dictionary with the information of the solution found (or empty if there is no solution).

- **Input parameters**:
  - `algorithm_name`: text string with the name of the algorithm to be executed (see supported algorithm list below).
  - `initial_state`: list representing the initial state of the puzzle.
  - `end_state`: list representing the goal state.
  - `size`: board size (e.g., `3` for 3x3 puzzles).
  - `depth`: maximum depth (only used in some algorithms, such as bounded DFS-B).

  - `heuristic_func`: optional heuristic function (**not used in this implementation**; heuristics are integrated according to the algorithm name).

- **Returns**:
  Dictionary including a single key (the name of the algorithm), having as a value a dictionary with:
  - `'path'`: list of states from the initial state to the goal.
  - `'cost'`: total cost of the solution.
  - `'depth'`: depth reached.
  - `'nodes_generated'`, `'nodes_expanded'`, `'time'`, `'max_nodes'`: performance metrics.

- **Supported algorithms**:
  - `'BFS'`: Breadth-first search (uniform cost without heuristics).
  - `'DFS-B'`: Bounded depth-first search (specific version with recursive control).
  - `'Voraz (Manhattan)'`: Greedy search using the Manhattan distance as a heuristic.
  - `'ID'`: Iterative deepening depth-first search (IDDFS).
  - `'A* (Manhattan)'`: A* algorithm using Manhattan heuristics.
  - `'A (MD + LC)'`: A* with improved herutistics, combining Manhattan and linear conflicts.
  - `'IDA* (Manhattan)'`: Iterative deepening A* with Manhattan heuristics.

`run_all_algorithms(initial_state, end_state, size, depth=50)`

This method **automatically** runs **all supported algorithms** on the same initial and goal states. It is used to compare their results.

- **Input parameters**:
  - `initial_state`: list representing the initial state of the puzzle.
  - `end_state`: list representing the goal state
  - `size`: board dimension.
  - `depth`: maximum depth (used in bounded algorithms such as DFS-B).

- **Returns**:
  Dictionary that associates the name of each algorithm with its corresponding result (same structure as that returned by `run_algorithm`).

- **Algorithms executed:**
  - `'BFS'`

- `'DFS-B'`
- `'Voraz (Manhattan)'`
- `'ID'`
- `'A* (Manhattan)'`
- `'A (MD + LC)'`
- `'IDA* (Manhattan)'`

`register_heuristic(name, heuristic, strategy='astar', include_in_all=True, replace=False)`

Registers a new heuristic so that it can be used directly through `run_algorithm` and, by default, also through `run_all_algorithms`.

- **Input parameters:**
  - `name`: label used to identify the heuristic.
  - `heuristic`: function that accepts (state, end_state) and returns its heuristic value.
  - `strategy`: strategy that the heuristic will use: 'astar', 'greedy', or 'ida'.
  - `include_in_all`: if True, the heuristic is automatically included in run_all_algorithms.
  - `replace`: when set to True, allows replacing an existing registration with the same name.

Representation of the states and execution of the algorithms

The states of the puzzle are represented as n x n matrices, which are converted into one-dimensional lists to be processed by the algorithms. Below is an example of code for a 3x3 puzzle:

```
size = 3  # 3x3 puzzle

# Initial and goal states as matrices
initial_state_matrix = [
    [7, 8, 1],
    [4, 0, 6],
    [2, 3, 5]
]

end_state_matrix = [
    [1, 2, 3],
    [8, 0, 4],
    [7, 6, 5]
]

# Conversion into one-dimensional lists
initial_state = [num for row in initial_state_matrix for num in row]
end_state = [num for row in end_state_matrix for num in row]
```

This code defines the size of the puzzle as 3x3 (`size = 3`) and represents both the initial state and the goal state in the form of matrices. It then converts these matrices into one-dimensional lists (`initial_state` and `end_state`), which is the format required by the search algorithms to process the puzzle states.

Next, the code checks whether the puzzle is solvable and executes all the algorithms:

```
results = {}
if isSolvable(initial_state, end_state):
    results = run_all_algorithms(initial_state, end_state, size, depth=40)
    show_results(results)
    plot_algorithm_comparison(results)
else:
    print("The puzzle is not solvable.")
```

This code fragment checks whether the initial state of the puzzle can be transformed into the goal state, using the function `isSolvable`. If so, it runs all search algorithms using `run_all_algorithms` and saves the results in the dictionary `results`. It then displays a summary of the metrics using `show_results` and a graphical comparison using `plot_algorithm_comparison`.

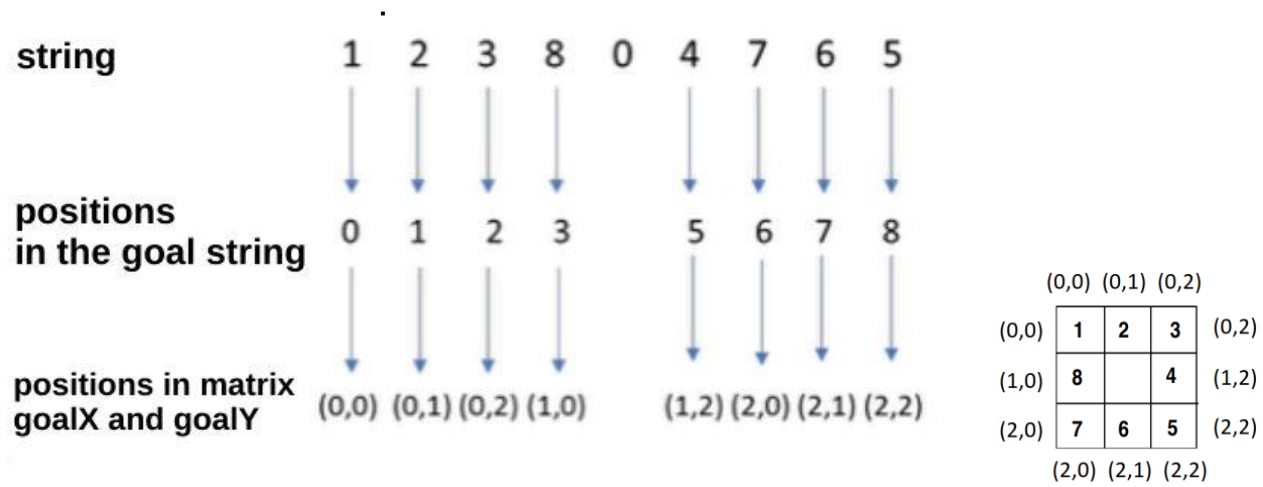
5. How to add a new heuristic

Let's suppose you want to add a function that implements an additional heuristic function. As an example, we explain here the `getManhattanDistance` function already included in the code:

```
def getManhattanDistance(state, end_state):  # state is the current state to be
                                             # evaluated, end_state is the goal state
  tot = 0
  size=int(math.sqrt(len(state)))
  for i in range(1, len(state)):       # goes through the n tiles
    goal = end_state.index(i) # position of tile i in the goal state (end_state)
    goalX = goal // size        # we turn the end_state from a string into a nxn matrix
    goalY = goal % size              # row (goalX) and column (goalY) of the tile
    idx = state.index(i)    # same operation but for the current row (idx) and column (idy)
    itemX = idx // size          # occupied by that tile in the current state
    itemY = idx % size
    tot += (abs(goalX - itemX) + abs(goalY - itemY)) # computing the distances
  return tot
```

The function receives in `state` the current state to be evaluated. For each one of the 8 tiles (the empty square is represented as 0, but it is not a tile), the distance from the current position of that tile to its position in the goal state (represented in the variable `end_state)` is computed. The intermediate operations are used to compute the positions of each tile in an nxn matrix (as in the puzzle) from the linear representation as a string.

For example, for the goal state (`end_state`), the transformation from the linear representation (string) to a matrix in a 3x3 puzzle would be:



Once the new heuristic function has been implemented, it must be registered using `register_heuristic`. It is not necessary to modify `run_algorithm` or `run_all_algorithms`.

```
def new_heuristic(state, end_state):
     # Computation of the heuristic value
     return value

register_heuristic('New Heuristic', new_heuristic)
```

The first argument is the label used to run and display the heuristic. The second is the newly implemented function. Since no strategy is specified, A* is used by default.

If you wish to use a different search strategy, it can be explicitly specified:

```
register_heuristic('New A*', new_heuristic, strategy='astar')
register_heuristic('New Greedy', new_heuristic, strategy='greedy')
register_heuristic('New IDA*', new_heuristic, strategy='ida')
```

By default, every registered heuristic is automatically included in run_all_algorithms. If you only want to execute it individually, use:

```
register_heuristic(
    'New Heuristic',
    new_heuristic,
    include_in_all=False
)
```

To run the new heuristic by itself, you can execute this:

```
# assuming that initial_state, end_state and size have been initialized

result = run_algorithm('New heuristic', initial_state, end_state, size)

show_results(result)
```

We can also run the new heuristic together with the rest of the algorithms:

```
# assuming that initial_state, end_state, size and max_depth have been
# initialized

results = run_all_algorithms(initial_state, end_state, size, max_depth)
show_results(results)
```

6. Work to be done by the student

This section outlines the task about the n-puzzle to be completed by each lab group (either one person or a group of two people maximum) must complete on the n-puzzle.

Students are asked to implement several heuristic functions and justify their behaviour. On the day of the lab exam, all students must upload their assignment:

- The Python code for each one of the heuristics (notebook files), including the answers and justifications for the questions asked.

In the case of a two-people lab group, both of them will upload the same files to their corresponding tasks. **NOTE: Do state clearly the names of both people in the text file containing the answers to the questions asked.**

1. Implement the **MISPLACED TILES** heuristic function seen in our theory lessons ($h(n)=W(n)$) for an A-type search.

2. Implement the **A* Euclidean** heuristic function: This is an A-type search that uses the Euclidean distance as a heuristic function; node expansion is performed according to the function $f(n)=g(n)+D(n)$, where $g(n)$ is the cost factor corresponding to the depth of node n in the search tree and $h(n)=D(n)$ is the Euclidean distance, i.e. the sum of the straight-line distances of each puzzle tile from its current position to the position it will occupy in the goal state (square root of the sum of the squares of the horizontal and vertical distances).

3. Implement the heuristic function **ROWS_COLUMNS** ($h(n)=RowCol(n)$) for an A-type search. This heuristic works as follows: for each tile on the board, if the tile is not in its correct goal row, 1 point is added; if the tile is not in its correct goal column, 1 point is added. Therefore, the minimum value of this heuristic for a tile is 0 (when the tile is correctly placed in its goal position), while the maximum value is 2 (when neither the row nor the column of the tile's position are equal to the value of the row and column in the goal position).

4. Implement the **A* Euclidean + LC** heuristic function

5. Implement a heuristic proposed by you as a combination of the above. It does not have to be A*. The idea is to try to propose a heuristic that you think works well.

6. For each heuristic, a comparative study with the other algorithms is required, including:

    - Determine which heuristics are admissible and consistent, justifying the answers.
    - To study admissibility, check that the heuristic equals 0 at the goal state, that it does not return negative values, and compare its value with the optimal solution cost across multiple examples. To study consistency, use the check_consistency function with different initial states. The absence of counterexamples provides empirical evidence but does not in itself constitute a proof. Propose the use of several examples to support the justifications of the answers.
    - Display results in tables such as the following to enable justifying answers using concrete data.

	BFS	DFS (Back)	Greedy	ID	A* Manh	A* MD + LC	A* Euclid	IDA* Manh	Misplaced	RowCol	A* Euclid + LC	Proposed
Nodes generated												
Nodes expanded												
Max nodes stored												
Solution cost												
Search depth												
Running time												

7. A detailed analysis study comparing the two best heuristics you have identified using the notebook "alg-compare-p3.ipynb", justifying your answers.

Appendix. How to use Jupyter Notebook or Google Colab

Option 1: Using Jupyter Notebook (recommended if working locally)

1. Make sure that you have Python and Jupyter Notebook installed. You can install it with:

```
pip install notebook
```

2. Go to the folder where the lab assignment files are located (search.py, puzzle-p1.ipynb, etc.).

3. Run the following command in the command line:

```
jupyter notebook
```

4. The browser will open. Click on the .ipynb file you want to use (for example, puzzle-p1.ipynb).

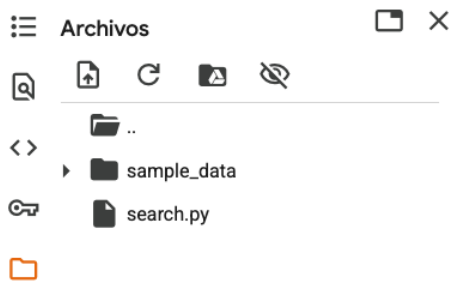
5. Run each cell with Shift + Enter.

Note: Make sure that the search.py file is in the same directory so that the functions are imported correctly.

Option 2: Using Google Colab

1. Go to https://colab.research.google.com

2. Upload the file "search.py" file through the “folder” icon > tab "Files" > "Upload". It should look like this image.



3. Open the intended notebook (e.g., puzzle-p2.ipynb) through “File” > “Upload notebook”.

4. Add, if it is not already there, a cell at the beginning of the notebook to import the file search.py:

```
from search import *
```

5. Run each cell with Shift + Enter.

Note: If you need to re-upload search.py after modifying it, restart the runtime environment (Runtime > Restart) and run it again.